SWEET SCOOP

Assignment 2 — Design: Services, Contracts & Schema

TEAM ID: 10

Team Leader: Annu Kumari (20251651024)

Team Members:

Prerna (20251651072)

Akanksha Upadhyay (20251651011)

Project Scope

Sweet Scoop is an online bakery and ice-cream ordering system. Customers browse products, manage their account and delivery address, place orders, make payments, track deliveries, and review products. Staff maintain the catalogue and stock. The design separates service-owned data and exposes operations through contracts.

Design rule: Each service owns its data store. Cross-service references are identifiers exchanged through APIs; database foreign keys do not cross service boundaries.

Document Contents

1. System capabilities
2. Service boundaries and responsibility
3. Service contracts
4. Full placeOrder specification
5. Five-property service validation
6. Database and diagram alignment notes

1. System Capabilities

The following distinct system jobs are the capability verbs used to derive service boundaries.

#	Capability	Description
1	Browse and search catalogue	*View product details, categories, flavours, tags, prices and availability.*
2	Manage products and stock	*Create or update product information and adjust inventory.*
3	Register and authenticate customers	Create customer accounts and establish authenticated sessions.
4	Maintain customer profiles and addresses	Read/update profile details and maintain delivery addresses.
5	Place and inspect orders	Submit item quantities, confirm totals and view order status/history.
6	Cancel orders	Cancel eligible orders and coordinate payment/inventory compensation.
7	Process and refund payments	Charge a payment method and issue eligible refunds.
8	Create and track deliveries	Create a shipment and expose its current status and event timeline.
9	Publish and read product reviews	Post ratings/comments and retrieve reviews and average ratings.
10	Report product popularity	Report best sellers and anonymized top buyers from order data.

Reporting note: Best-seller and anonymized top-buyer reporting is a stated capability. The exact reporting endpoint and aggregation implementation should be specified in the service diagram/API artefacts if it is in the assessed implementation scope.

2. Service Boundary Benchmark

The proposed design has six services. The provided project brief does not assign named owners to each service, so implementation ownership must be confirmed with the team. Akanksha's database/schema responsibility is identified separately rather than inventing service assignments.

Service	Data owned	Dependencies / calls	Responsibility
Catalogue	Categories, product descriptions, prices, images, tags, inventory	No synchronous dependency required for core catalogue operations	Service owner: to confirm; DB/schema work: Akanksha
Customer	Customer profile, address book, password hashes / identity metadata	No synchronous dependency required for core account operations	Service owner: to confirm; DB/schema work: Akanksha
Order	Orders, order-line snapshots, totals, coupon definitions/usage	Customer, Catalogue, Payment, Delivery	Service owner: to confirm; DB/schema work: Akanksha
Payment	Payment attempts, provider references, refund transactions	External payment provider or simulator	Service owner: to confirm; DB/schema work: Akanksha
Delivery	Delivery records, tracking references, status timeline	External delivery partner (SOAP/API adapter)	Service owner: to confirm; DB/schema work: Akanksha
Reviews	Review documents, ratings and review metadata	May query Catalogue API for product validation	Service owner: to confirm; MongoDB/schema work: Akanksha

Boundary constraint: Order may store a product ID and price/name snapshot, but must not create a foreign key into the Catalogue database. Payment and Delivery keep their own records and are referenced by IDs from Order.

3. Consumer-Facing Service Contracts

Contracts specify operation, input, output and externally visible errors. They intentionally hide database tables, SQL, internal ID formats and implementation details.

3.1 Catalogue Service

Operation	Input	Output	Errors
List products	Optional category, flavour, tag, search term, cursor and limit	Product summaries and next-page cursor	400 invalid query; 422 invalid filter
Get product	Product resource identifier	Product detail and cache validator	404 product not found
Create product	Name, category, price; optional tags, image and initial stock	Created product representation	401 unauthorised; 409 conflict; 422 invalid product
Replace product	Full product representation; optional If-Match validator	Updated product and new ETag	401; 404; 412 stale version; 422 invalid product
Adjust inventory	Stock delta and reason	Current stock level	401; 404; 409 insufficient stock; 422 invalid adjustment

3.2 Customer Service

Operation	Input	Output	Errors
Register customer	Email, password, full name, optional phone	Created profile summary	409 email already used; 422 invalid profile
Authenticate customer	Email and password	Bearer token and profile summary	401 invalid credentials; 422 invalid request
Get/update profile	Authenticated profile fields	Current or updated name, email and phone	401 unauthenticated; 403 not owner; 404; 409 email conflict; 422 invalid profile
List addresses	Authenticated customer context	Address summaries	401 unauthenticated; 403 not owner
Add address	Label, street, city, state, postal code, country	Created address representation	401; 403 not owner; 422 invalid address

3. Consumer-Facing Service Contracts (continued)

3.3 Order Service

Operation	Input	Output	Errors
Place order	Authenticated customer, product/quantity lines, owned address, payment method, optional coupon, idempotency key	Confirmed order, totals, payment and delivery references	400 malformed request; 401; 402 payment declined; 409 idempotency/state conflict; 422 invalid item/address/stock/coupon; 429 throttled; 502 dependency failure
Get order	Authenticated order resource	Order, line items, totals and current status	401; 404 absent or not owned
List customer orders	Authenticated customer; optional cursor and limit	Order summaries and next cursor	401; 403 not owner; 422 invalid pagination
Cancel order	Authenticated order resource	Cancelled status and refund result	401; 404; 409 not cancellable; 502 compensation/dependency failure

3.4 Payment Service

Operation	Input	Output	Errors
Process payment	Order reference, customer reference, amount, payment method, idempotency key	Payment status and payment reference	402 declined; 409 duplicate/conflict; 422 invalid payment
Get payment	Payment resource reference	Payment status and amount	404 payment not found
Refund payment	Payment reference and idempotency key	Refund status and reference	404; 409 already refunded/not refundable; 502 provider failure

3.5 Delivery Service

Operation	Input	Output	Errors
Create delivery	Confirmed order reference, address snapshot, item summary, idempotency key	Delivery record and tracking reference	409 duplicate/conflict; 422 unsupported address; 502 partner unavailable
Track delivery	Delivery or order reference	Current status and ordered event timeline	404 delivery not found
Update delivery status	Delivery reference and legal next status	Updated status and event	404; 409 illegal transition; 422 invalid status

3. Consumer-Facing Service Contracts (continued)

3.6 Reviews Service

Operation	Input	Output	Errors
Post review	Product reference, authenticated customer, rating, optional comment	Created review	401 unauthenticated; 403 not author; 422 invalid rating/content
List reviews	Product reference and optional cursor	Page of reviews and next cursor	422 invalid cursor; 503 review store unavailable
Get average rating	Product reference	Average score and review count	503 review store unavailable

API Consistency Requirements

- Use authentication on customer-specific operations and enforce ownership on every read, update and cancel request.
- Use idempotency keys for payment, refund and order creation to prevent duplicate side effects on retries.
- Return stable, documented error categories; do not expose stack traces, database errors, provider secrets or internal service URLs.
- Use pagination for potentially large catalogue, order and review lists.

4. Full Specification: placeOrder

Operation: POST /v1/orders **Purpose:** Create an order and coordinate payment, stock reservation and delivery across independently owned services.

Input

- Authenticated bearer token; the customer identifier must match the token subject.
- One or more items, each containing productId and quantity (integer ≥ 1).
- Delivery address owned by the authenticated customer.
- Supported payment method; optional active coupon.
- Required Idempotency-Key header.

Successful Output

Return 201 Created, a Location header (/v1/orders/{orderId}), and an order representation containing order status, line-item snapshots, subtotal, discount, tax, total, payment reference, delivery reference/tracking link and creation timestamp. Repeating the same request with the same idempotency key returns the original result and does not charge again or reserve stock twice.

Processing Sequence

- Validate request and authenticate the customer.
- Verify that the delivery address belongs to that customer.
- Retrieve product details and check availability through Catalogue.
- Calculate totals using currency-safe, cent-rounded arithmetic and persist a pending order with item snapshots.
- Request payment from Payment using the idempotency key.
- Reserve the required stock through Catalogue.
- Request delivery creation from Delivery.
- Persist returned service references, mark the order confirmed and return the response.

Failure Handling / Compensation

If payment fails, mark the order failed. If a later stock-reservation or delivery step fails after payment succeeds, request a refund and release only stock that was successfully reserved. Mark the order failed, log the internal cause and return a safe dependency error. Compensation is best-effort; independent services do not participate in one atomic distributed database transaction.

Error Cases

400 malformed JSON/request; 401 missing or invalid token; 402 payment declined; 409 duplicate-key misuse or incompatible order state; 422 empty/invalid items, invalid address, insufficient stock, invalid coupon or unsupported payment method; 429 rate limit; 502 required downstream service failed.

Hidden from Callers

Service URLs and topology, database tables and SQL, product-price lookup internals, calculation implementation, SOAP envelope and delivery-partner vocabulary, retry timing, circuit-breaker state, provider credentials, and compensation mechanics.

5. Validation Against the Five Service Properties

The table checks whether each proposed service is reachable, self-contained, governed by a contract, independently deployable and loosely coupled. “Partial” indicates a known dependency or operational condition that needs controls.

Service	Reachable	Self-contained	Contract	Independent	Loosely coupled	Gap / remedy
Catalogue	Yes	Yes	Yes	Yes	Yes	Keep product/category/inventory data within Catalogue. Publish versioned APIs for product lookup and stock adjustment.
Customer	Yes	Yes	Yes	Yes	Yes	Store password hashes only; protect JWT signing secrets and enforce customer ownership.
Order	Yes	Yes	Yes	Yes	Partial	Orchestrates Payment, Catalogue and Delivery. Use timeouts, bounded retries, idempotency and compensation.
Payment	Yes	Yes	Yes	Yes	Yes	Provider integration is assumed to be simulated; production needs a compliant payment provider and secret management.
Delivery	Yes	Yes	Yes	Yes	Partial	External carrier is assumed to be a stub/adapter; configure a real endpoint, credentials and retry policy for production.
Reviews	Yes	Yes	Yes	Yes	Yes	MongoDB store is isolated. If product/customer validation calls are required, use APIs rather than cross-database joins.

6. Database and Diagram Alignment Notes

The ER diagram and SQL sketch should mirror the ownership table above. Suggested relational ownership: Catalogue owns categories/products/inventory; Customer owns profiles/addresses; Order owns orders/order items/coupon data; Payment owns payment/refund records; Delivery owns delivery/status events. Reviews owns review documents in MongoDB.

Cross-service IDs: productId, customerId, orderId, paymentId and deliveryId are API-level references, not cross-database foreign keys. Internal foreign keys are allowed only between tables owned by the same service. Do not store raw passwords or payment-card details.

Assumptions Requiring Team Confirmation

- Service-level implementation owners are not assigned in the provided project brief; confirm these names with the team before final submission.
- The exact best-seller/top-buyer reporting API and source-of-truth ownership should be reflected in services.drawio if implemented.
- Ensure services.drawio, schema.drawio and schema.sql use the same service names, operations and data ownership shown in this document.